Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development and Jira

Roll No.: T013	Name: Saqlain Khan
Class: TYIT	Batch: 1
Date of Assignment: 05-08-2026	Date/Time of Submission: 05-08-2026

PRACTICAL 7

AIM :- Machine Learning

2) Using a suitable regression dataset, implement Linear Regression using scikit-Learn.

CODE :-

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

data = pd.read_csv("student_marks.csv")

X = data[['Hours']]

y = data['Marks']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

model = LinearRegression()

model.fit(X_train, y_train)

print("Training Accuracy : ",

round(model.score(X_train, y_train) * 100, 2))

print("Testing Accuracy : ",

round(model.score(X_test, y_test) * 100, 2))

new_data = pd.DataFrame([[11]],columns=['Hours'])

pred = model.predict(new_data)
```

print("Predicted Marks for 11 Hours : ",round(pred[0],2))

print("Saqlain Khan T013")

OUTPUT :-

```
Training Accuracy :  99.64
Testing Accuracy :  98.71
Predicted Marks for 11 Hours :  93.57
Saqlain Khan T013
```

3) Using the MNIST dataset (digit vs not digit), implement a binary classifier.

CODE :-

from sklearn.datasets import load_digits

from sklearn.linear_model import LinearRegression

from sklearn.linear_model import LogisticRegression

data = pd.read_csv("mnist_test.csv")

digits = load_digits()

X = digits.data

y = digits.target

y = (y == 5)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

model = LogisticRegression(max_iter = 1000)

model.fit(X_train, y_train)

print("Training Accuracy : ", model.score(X_train, y_train)*100)

print("Testing Accuracy : ", model.score(X_test, y_test)*100)

prediction = model.predict([X_test[0]])

if prediction[0]:

print("Digit is 5")

else:

print("Digit is not 5")

print("Saqlain Khan T013")

OUTPUT :-

```
Training Accuracy :  100.0
Testing Accuracy :  98.88888888888889
Digit is not 5
Saqlain Khan T013
```

PART 2 :

1. California Housing Dataset

Problem Statement

A real estate company wants to predict the median house value based on housing features.

Features :-

• Median Income

• House Age

• Average Rooms

• Average Bedrooms

• Population

• Average Occupancy

• Latitude

• Longitude

Target

Median House Value

Tasks

1. Load the dataset using Pandas.

2. Display the first five records.

3. Check for missing values.

4. Select one suitable feature (e.g., Median Income) as the independent variable.

5. Split the dataset into training and testing sets.

6. Train a Linear Regression model.

7. Predict the house prices.

8. Display:

o Coefficient

o Intercept

o R2 Score

9. Plot the regression line.

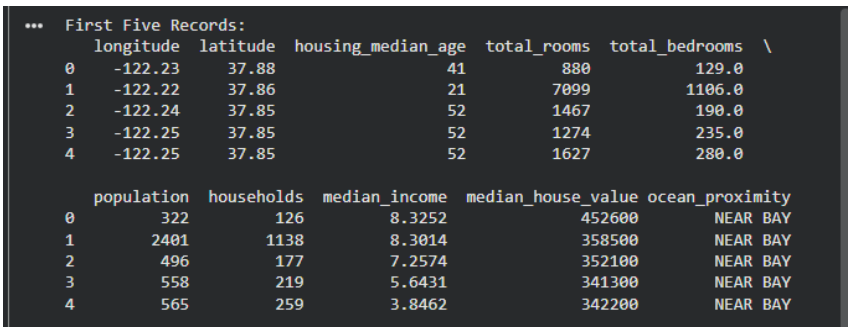
10. Interpret the results.

CODE :-

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
df = pd.read_csv("housing.csv")
print("First Five Records:")
print(df.head())
print("\nMissing Values:")
print(df.isnull().sum())
X = df[['median_income']]
y = df['median_house_value']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

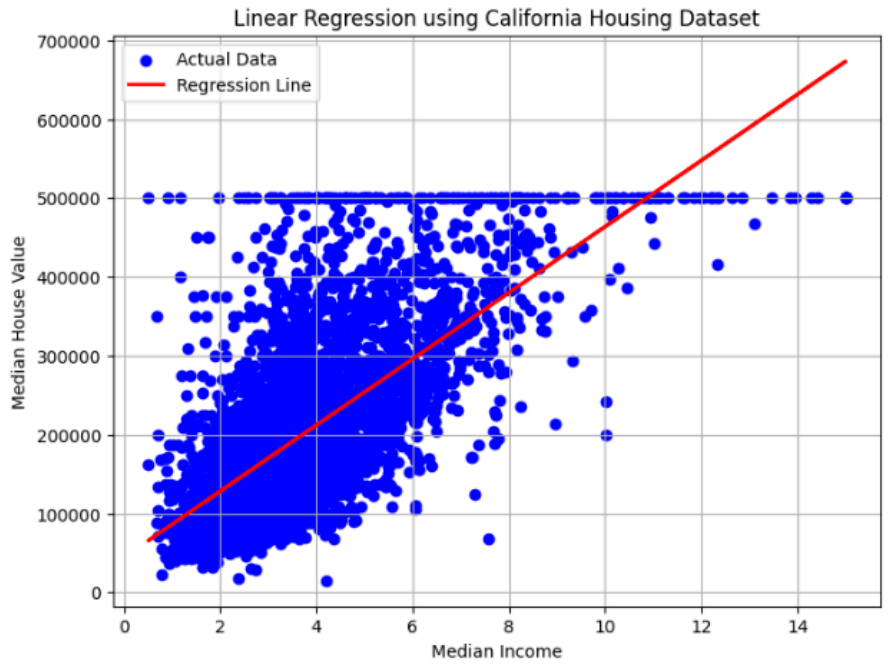
```
)

model = LinearRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nCoefficient:", model.coef_[0])

print("Intercept:", model.intercept_)

print("R2 Score:", r2_score(y_test, y_pred))

plt.figure(figsize=(8,6))

plt.scatter(X_test, y_test, color='blue', label='Actual Data')

plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression Line')

plt.xlabel("Median Income")

plt.ylabel("Median House Value")

plt.title("Linear Regression using California Housing Dataset")

plt.legend()

plt.grid(True)

plt.show()
```

OUTPUT :-



```
First Five Records:
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -122.23     37.88                  41          880           129.0
1    -122.22     37.86                  21         7099          1106.0
2    -122.24     37.85                  52         1467           190.0
3    -122.25     37.85                  52         1274           235.0
4    -122.25     37.85                  52         1627           280.0

   population  households  median_income  median_house_value ocean_proximity
0         322         126         8.3252              452600        NEAR BAY
1        2401        1138         8.3014              358500        NEAR BAY
2         496         177         7.2574              352100        NEAR BAY
3         558         219         5.6431              341300        NEAR BAY
4         565         259         3.8462              342200        NEAR BAY
```

```
Missing Values:
longitude               0
latitude                0
housing_median_age      0
total_rooms             0
total_bedrooms        207
population              0
households              0
median_income           0
median_house_value      0
ocean_proximity         0
dtype: int64

Coefficient: 41933.84939381272
Intercept: 44459.72916907875
R2 Score: 0.45885918903846656
```




```
Saqlain Khan T013
```

PART 3 :

Q.1 Modify the program to detect Digit 7 vs Not Digit 7 instead of Digit 5 vs Not Digit 5.

CODE :-

```
from sklearn.datasets import load_digits

from sklearn.linear_model import LinearRegression

from sklearn.linear_model import LogisticRegression

data = pd.read_csv("mnist_test.csv")

digits = load_digits()

X = digits.data

y = digits.target
```

```
y = (y == 7)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

model = LogisticRegression(max_iter = 1000)

model.fit(X_train, y_train)

print("Training Accuracy : ", model.score(X_train, y_train)*100)

print("Testing Accuracy : ", model.score(X_test, y_test)*100)

prediction = model.predict([X_test[0]])

if prediction[0]:

print("Digit is 7")

else:

print("Digit is not 7")

print("Saqlain Khan T013")
```

OUTPUT :-

```
Training Accuracy :  100.0
Testing Accuracy :  99.07407407407408
Digit is not 7
Saqlain Khan T013
```

Q.2 Modify the program to detect Digit 0 vs Not Digit 0.

CODE :-

```
from sklearn.datasets import load_digits

from sklearn.linear_model import LinearRegression

from sklearn.linear_model import LogisticRegression

data = pd.read_csv("mnist_test.csv")

digits = load_digits()

X = digits.data

y = digits.target
```

```
y = (y == 0)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
model = LogisticRegression(max_iter = 1000)
model.fit(X_train, y_train)
print("Training Accuracy : ", model.score(X_train, y_train)*100)
print("Testing Accuracy : ", model.score(X_test, y_test)*100)
prediction = model.predict([X_test[0]])
if prediction[0]:
print("Digit is 0")
else:
print("Digit is not 0")
print("Saqlain Khan T013")
```

OUTPUT :-

```
Training Accuracy :  100.0
Testing Accuracy :  99.44444444444444
Digit is not 0
Saqlain Khan T013
```

Q.3 Modify the program to detect Digit 9 vs Not Digit 9.

CODE :-

```
from sklearn.datasets import load_digits
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
data = pd.read_csv("mnist_test.csv")
digits = load_digits()
X = digits.data
y = digits.target
```

```
y = (y == 9)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

model = LogisticRegression(max_iter = 1000)

model.fit(X_train, y_train)

print("Training Accuracy : ", model.score(X_train, y_train)*100)

print("Testing Accuracy : ", model.score(X_test, y_test)*100)

prediction = model.predict([X_test[0]])

if prediction[0]:

print("Digit is 9")

else:

print("Digit is not 9")

print("Saqlain Khan T013")
```

OUTPUT :-

```
Training Accuracy :  99.68178202068417
Testing Accuracy :  98.51851851851852
Digit is not 9
Saqlain Khan T013
```

Q.4 Modify the program to display the first 20 predictions instead of only one prediction.

CODE :-

```
from sklearn.datasets import load_digits

from sklearn.linear_model import LinearRegression

from sklearn.linear_model import LogisticRegression

data = pd.read_csv("mnist_test.csv")

digits = load_digits()

X = digits.data

y = digits.target
```

```
y = (y == 9)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

print("Training Accuracy : ", model.score(X_train, y_train) * 100)

print("Testing Accuracy : ", model.score(X_test, y_test) * 100)

predictions = model.predict(X_test[:20])

for prediction in predictions:

if prediction:

print("Digit is 9")

else:

print("Digit is not 9")

print("Saqlain Khan T013")
```

OUTPUT :-